

OpenAPI / Swagger

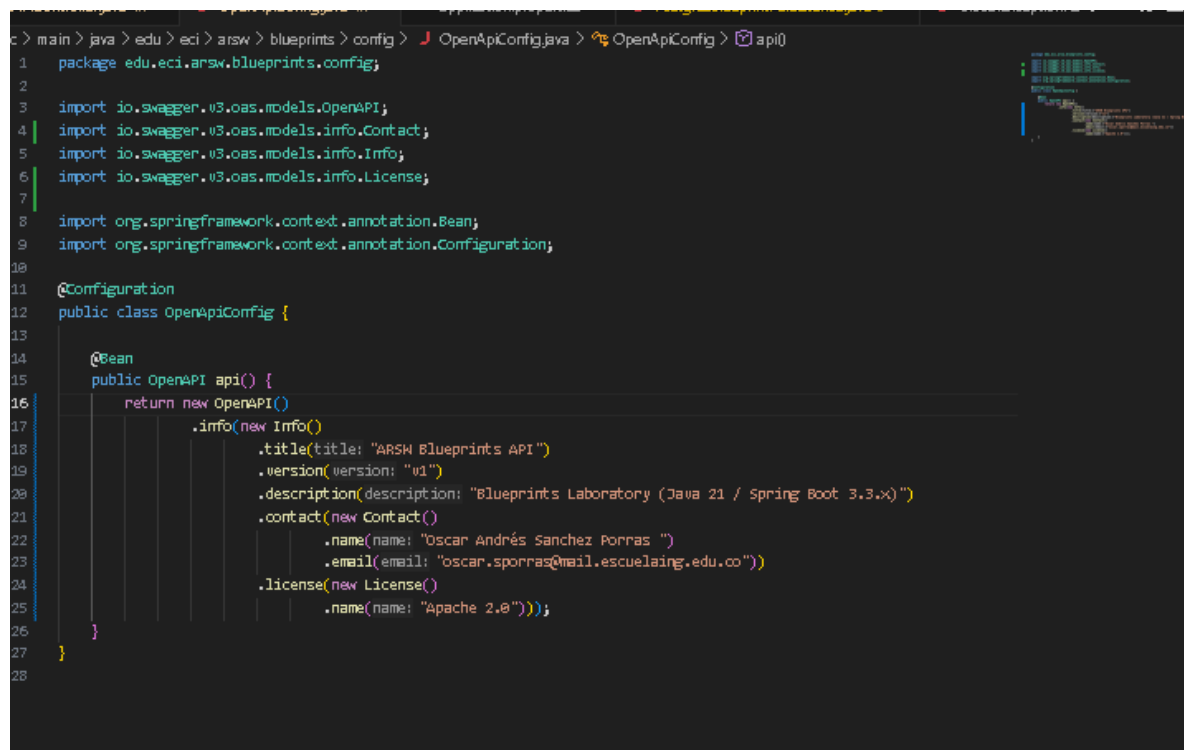
Objetivo

Configurar documentación automática de la API REST usando **Springdoc OpenAPI**, permitiendo:

- Generación automática de especificación OpenAPI 3
- Visualización interactiva en Swagger UI
- Documentación profesional de endpoints
- Visualización de modelos y códigos HTTP

Configuración global OpenAPI

Se actualizo la siguiente clase



```
1 package edu.eci.arsw.blueprints.config;
2
3 import io.swagger.v3.oas.models.OpenAPI;
4 import io.swagger.v3.oas.models.info.Contact;
5 import io.swagger.v3.oas.models.info.Info;
6 import io.swagger.v3.oas.models.info.License;
7
8 import org.springframework.context.annotation.Bean;
9 import org.springframework.context.annotation.Configuration;
10
11 @Configuration
12 public class OpenApiConfig {
13
14     @Bean
15     public OpenAPI api() {
16         return new OpenAPI()
17             .info(new Info()
18                 .title(title: "ARSW Blueprints API")
19                 .version(version: "v1")
20                 .description(description: "Blueprints Laboratory (Java 21 / Spring Boot 3.3.x)")
21                 .contact(new Contact()
22                     .name(name: "Oscar Andrés Sanchez Porras ")
23                     .email(email: "oscar.sporras@mail.escuelaing.edu.co"))
24                 .license(new License()
25                     .name(name: "Apache 2.0")));
26     }
27 }
28
```

Cambios realizados

- Se definió título de la API
- Se definió versión (v1)
- Se agregó descripción

- Se agregó información de contacto
- Se agregó licencia

URLs disponibles automáticamente

Después de levantar la aplicación:

`http://localhost:8080/swagger-ui.html`

También funciona:

`http://localhost:8080/swagger-ui/index.html`

Y el JSON:

`http://localhost:8080/v3/api-docs`

Documentación del controlador

Se modifíco el controlador agregando:

- @Tag
- @Operation
- @ApiResponses
- @ApiResponse

Se modifica el controller para que la API esté bien documentada y sea más profesional, sin alterar su funcionamiento.

```

@Tag(name = "Blueprints", description = "Operations related to blueprints")
@RestController
@RequestMapping("/api/v1/blueprints")
public class BlueprintsApiController {

    private final BlueprintsServices services;

    public BlueprintsApiController(BlueprintsServices services) {
        this.services = services;
    }

    // GET /blueprints
    @Operation(summary = "Get all blueprints", description = "Returns the complete list of blueprints stored in the database")
    @ApiResponse(responseCode = "200", description = "Blueprints retrieved successfully")
    @GetMapping
    public ResponseEntity<ApiResponse<Set<Blueprint>>> getAll() {

        Set<Blueprint> data = services.getAllBlueprints();

        return ResponseEntity.ok(
            new ApiResponse<>>(200, "execute ok", data));
    }

    // GET /blueprints/{author}
    @Operation(summary = "Get blueprints by author", description = "Returns all blueprints created by the specified author")
    @ApiResponse(responseCode = "200", description = "Blueprints retrieved successfully")
    @ApiResponse(responseCode = "404", description = "Author not found")
    @GetMapping("/{author}")
    public ResponseEntity<ApiResponse<Set<Blueprint>>> byAuthor(@PathVariable String author) {

        try {
            Set<Blueprint> data = services.getBlueprintsByAuthor(author);

            return ResponseEntity.ok(
                new ApiResponse<>>(200, "execute ok", data));
        } catch (BlueprintNotFoundException e) {

            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(new ApiResponse<>>(404, e.getMessage(), null));
        }
    }

    // GET /blueprints/{author}/{bpname}
    @Operation(summary = "Get blueprint by author and name", description = "Returns a specific blueprint identified by author and name")
    @ApiResponse(responseCode = "200", description = "Blueprint retrieved successfully")
    @ApiResponse(responseCode = "404", description = "Blueprint not found")
    @GetMapping("/{author}/{bpname}")
    public ResponseEntity<ApiResponse<Blueprint>> byAuthorAndName(
        @PathVariable String author,
        @PathVariable String bpname) {

        try {

```

```

        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiResponse<>(404, e.getMessage(), null));
    }
}

// POST /blueprints
@Operation(summary = "Create a new blueprint", description = "Creates a new blueprint with the provided author, name and points")
@ApiResponses({
    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "201", description = "Blueprint created successfully", headers = {"X-Blueprint-Id": "The ID of the created blueprint"}),
    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "403", description = "Blueprint already exists")
})
@PostMapping
public ResponseEntity<ApiResponse<Void>> add(
    @Valid @RequestBody NewBlueprintRequest req) {

    try {

        Blueprint bp = new Blueprint(req.author(), req.name(), req.points());
        services.addNewBlueprint(bp);

        return ResponseEntity.status(HttpStatus.CREATED)
            .body(new ApiResponse<>(201, "Blueprint created", null));

    } catch (BlueprintPersistenceException e) {

        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(new ApiResponse<>(403, e.getMessage(), null));

    }
}

// PUT /blueprints/{author}/{bpname}/points
@Operation(summary = "Add a point to a blueprint", description = "Adds a new point (x,y) to an existing blueprint")
@ApiResponses({
    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "202", description = "Point added successfully", headers = {"X-Blueprint-Id": "The ID of the blueprint"}),
    @io.swagger.v3.oas.annotations.responses.ApiResponse(responseCode = "404", description = "Blueprint not found")
})
@PutMapping("/{author}/{bpname}/points")
public ResponseEntity<ApiResponse<Void>> addPoint(
    @PathVariable String author,
    @PathVariable String bpname,
    @RequestBody Point p) {

    try {

        services.addPoint(author, bpname, p.x(), p.y());

        return ResponseEntity.status(HttpStatus.ACCEPTED)
            .body(new ApiResponse<>(202, "Point added", null));

    } catch (BlueprintNotFoundException e) {

        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiResponse<>(404, e.getMessage(), null));

    }
}
}

```

Esquemas Automáticos Generados

Springdoc detectó automáticamente los siguientes modelos:

- Blueprint

- Point
- ApiResponse<T>
- NewBlueprintRequest

Estos aparecen en la sección **Schemas** de Swagger UI.

Swagger
Supported by SMARTBEAR

/v3/api-docs Explore

ARSW Blueprints API v1 OAS 3.0

[/v3/api-docs](#)

Blueprints Laboratory (Java 21 / Spring Boot 3.3.x)

Contact [Oscar Andrés Sanchez Porras](#)

Apache 2.0

Servers

[http://localhost:8080 - Generated server url](#)

Blueprints

Operations related to blueprints

PUT	/api/v1/blueprints/{author}/{bpname}/points	Add a point to a blueprint	▼
GET	/api/v1/blueprints	Get all blueprints	▼
POST	/api/v1/blueprints	Create a new blueprint	▼
GET	/api/v1/blueprints/{author}	Get blueprints by author	▼
GET	/api/v1/blueprints/{author}/{bpname}	Get blueprint by author and name	▼

PUT

/api/v1/blueprints/{author}/{bpname}/points

Add a point to a blueprint

^

Adds a new point (x,y) to an existing blueprint identified by author and name

Parameters

Try it out

Name	Description
author * required string (path)	author
bpname * required string (path)	bpname

Request body

required

application/json

^

Example Value

Schema

GET

/api/v1/blueprints

Get all blueprints

^

Returns the complete list of blueprints stored in the system

POST

/api/v1/blueprints

Create a new blueprint

Creates a new blueprint with the provided author, name and list of points

GET

/api/v1/blueprints/{author}

Get blueprints by author

Returns all blueprints created by the specified author

GET

/api/v1/blueprints/{author}/{bpname}

Get blueprint by author and name

Returns a specific blueprint identified by its author and name